Surgically oxidising the Zephyr build system

for 100x performance improvements

Armin Brauns

About me

At work:

- Embedded software engineer at *SILA Embedded Solutions GmbH*
- Usually somewhere between C and voltage levels
- No embedded Rust - so far!

In my own time:

- Rust is a great language to have fun with!
- Most hobby projects are at a higher level - web services etc.

The zephyr build system

- Unified development experience for a ton of different environments (architecures, hardware, application scales, ...)
- Comes at a cost:
  - ≈30 000 lines of CMake
  - ≈20 000 lines of python
  - sometimes, ridiculously long build times:

```
$ west build --pristine [...]
$ touch src/main.c
$ time west build
[...]
Executed in   36.53 secs
   usr time   34.68 secs
   sys time    2.46 secs
```

SILA EMBEDDED SOLUTIONS

Digression: userspace (`CONFIG_USERSPACE`)

- Allows running threads with reduced permissions
- Enforced by Memory Protection Unit (MPU) and software checks in system calls
- Permissions are granted per kernel object (mutexes, pipes, threads, devices, …)

SILA EMBEDDED SOLUTIONS

Kobject metadata

Each kernel object has a bit of metadata:

```
struct k_object {
  void *name;
  uint8_t perms[CONFIG_MAX_THREAD_BYTES];
  uint8_t type;
  uint8_t flags;
  union k_object_data data;
};
```

⟹ Used to validate kernel objects passed as syscall parameters by userspace

Kobject metadata (ii)

- Metadata needs to live in trusted kernel memory
- Kobjects may be anywhere in memory
- Fast mapping from kobject address to metadata is needed

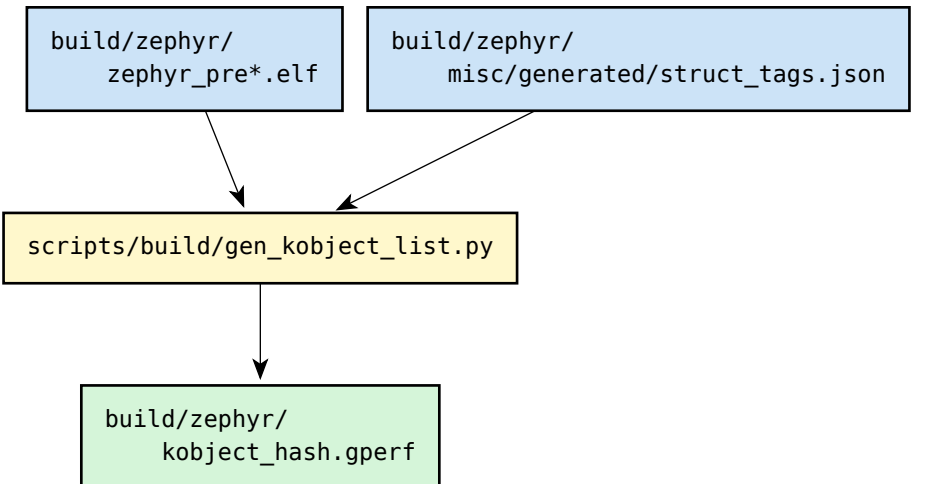
Kobject metadata (iii)

Solution:

1. Kobject instances are extracted from the ELF binary's debug information
2. Metadata is created and indexed using a Perfect Hash Function
3. Indexed metadata is compiled back into ELF

More information: `doc/kernel/usermode/kernelobjects.rst`

gen_kobject_list.py



`gen_kobject_list.py` (ii)

`struct_tags.json` input file contains names of network and API structs:

```
{
    "__net_socket": [
        "modem_socket",
        "net_context",
        "quic_stream",
        "quic_context",
        "tls_context",
        "websocket_context",
        [...]
    ],
    "__subsystem": [
        "gpio_driver_api",
        "reset_driver_api",
        [...]
```

`gen_kobject_list.py` (iii)

`kobject_hash.gperf` output file contains metadata of detected kobjects:

```
[...]
"\x7c\x11\x18\x10", {0}, K_OBJ_STACK,
    0 | K_OBJ_FLAG_INITIALIZED, { .unused = 0 }

"\x94\x11\x18\x10", {0}, K_OBJ_MSGQ,
    0 | K_OBJ_FLAG_INITIALIZED, { .unused = 0 }

"\x78\x14\x18\x10", {0}, K_OBJ_THREAD, 0, { .thread_id = 1 }

"\x28\xcc\x00\x18", {0}, K_OBJ_DRIVER_GPIO,
    0 | K_OBJ_FLAG_DRIVER, { .unused = 0 }

"\x44\xcc\x00\x18", {0}, K_OBJ_DRIVER_GPIO,
    0 | K_OBJ_FLAG_DRIVER, { .unused = 0 }
[...]
```

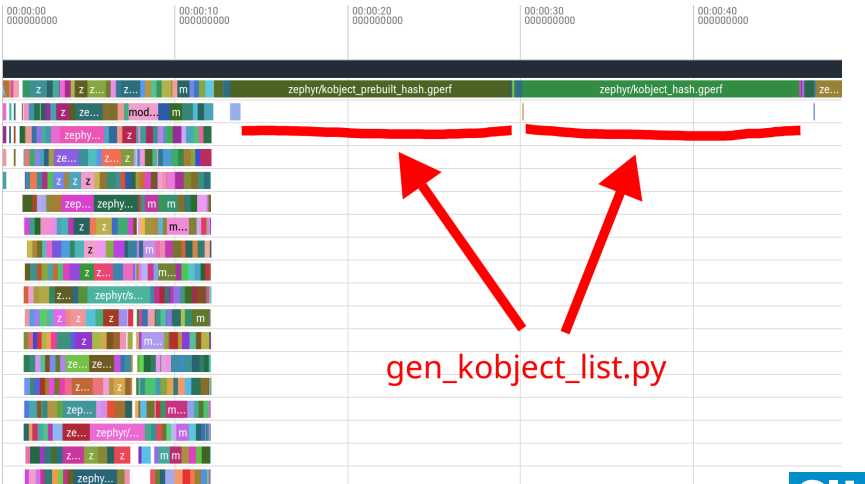
Benchmarking

Ninja outputs precise timestamps for every build task, out of the box!

- Converted into Chrome tracing format using https://github.com/nico/ninjatracing
- Loaded into https://ui.perfetto.dev/

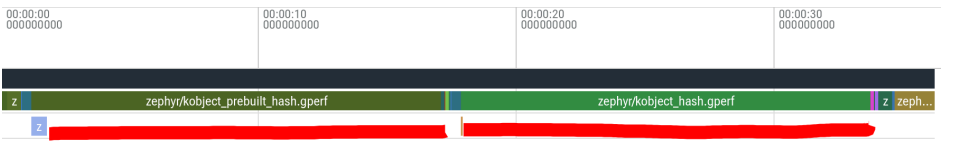
Building a decently-sized application


```
$ west build --pristine [...]
```



Even worse: incremental build


```
$ touch src/main.c
$ west build
```



36s build time, 31s spent waiting on `gen_kobject_list.py`!

This is extremely slow!
Let's rewrite it in Rust.

The rewrite

https://github.com/arbrauns/gen_kobject_list

- approx. 1500 lines of code
- mostly ported 1:1, line-by-line
  - some small idiomatic changes
  - more library use for DWARF debug info
- 100% drop-in replacement for python script
  - automatically detected by upstream build system

Results


```
$ cargo install gen_kobject_list
$ west build --pristine [...]
$ touch src/main.c
$ time west build
[...]
Executed in    5.21 secs
   usr time    4.54 secs
   sys time    1.25 secs
```

Finally, back to useful iteration times!

...before

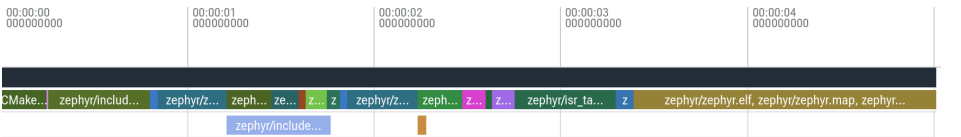

```
$ west build --pristine [...]
$ touch src/main.c
$ time west build
[...]
Executed in   36.53 secs
   usr time   34.68 secs
   sys time    2.46 secs
```

Benchmarks


```
$ west build --pristine [...]
```

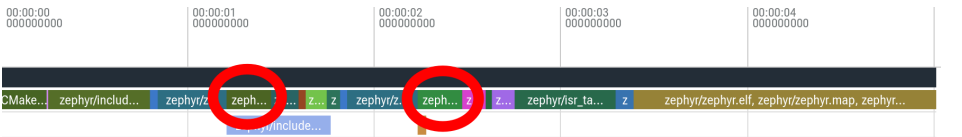
Benchmarks (ii)


```
$ touch src/main.c
$ west build
```



Benchmarks (iii)

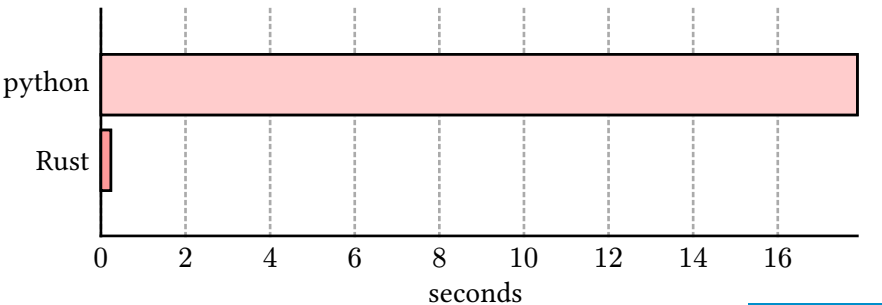

```
$ touch src/main.c
$ west build
```



Only 200ms spent blocked on `gen_kobject_list`!

Tests

- `test.sh` script in repository allows comparing Rust and python implementations
- Except for some small formatting differences, outputs should be bit-for-bit identical!
- Also useful for more precise performance comparison:



SILA EMBEDDED SOLUTIONS

The end

If you are annoyed by long `CONFIG_USERSPACE=y` builds, give it a try!

https://github.com/arbrauns/gen_kobject_list

Questions?

Presented using typst + diatypst

Armin Brauns
armin.brauns@embedded-solutions.at

SILA EMBEDDED SOLUTIONS